

MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DA EDUCAÇÃO MÉDIA E TECNOLÓGICA
INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA FARROUPILHA
Campus São Borja - RS



Técnico em Informática Integrado

Programação III

Ana Júlia Amarilho

Questões - 25/05/2026

São Borja, 27/02/2026

- 1) Encapsulamento é proteger os dados de uma classe, permitindo acesso somente por métodos controlados. public: pode ser acessado de qualquer lugar. protected: pode ser acessado pela própria classe e pelas classes filhas. private: só pode ser acessado dentro da própria classe. Não é boa prática deixar tudo public, porque qualquer parte do código pode alterar os dados sem controle, causando erros e falta de segurança.
- 2) Usamos getAtributo() para acessar atributos privados de forma controlada. O método set pode validar informações antes de salvar. Assim evitamos salvar valores inválidos.
- 3) “Um atributo private pode ser lido diretamente fora da classe.” - F
Porque private só funciona dentro da classe.
“Getters e setters só fazem sentido quando os atributos são private.” - V
Eles controlam o acesso aos atributos privados.
“Encapsulamento aumenta a segurança e a manutenção do código.” - V
Porque protege os dados e organiza melhor o sistema.
“É possível ter um getter sem ter um setter para o mesmo atributo.” - V
O atributo pode ser apenas leitura.
- 4) linha 14; echo \$p->nome; O atributo é private.

```
<?php
class Produto {
private $nome;
private $preco;
public function __construct($nome, $preco) {
$this->nome = $nome;
$this->preco = $preco;
}
public function getNome(){
return $this->nome;
}}
$p = new Produto('Teclado', 150.00);
echo $p->getNome(); ?>
```

- 5) Erro linha 13: echo 'Aluno: ' . \$nome . ' - Nota: ' . \$nota; As variáveis da classe precisam de \$this->.

```
<?php
class Aluno {
private $nome;
private $nota;
public function __construct($nome, $nota) {
$this->nome = $nome;
$this->nota = $nota;}
public function exibir() {
echo 'Aluno: ' . $this->nome . ' - Nota: ' . $this->nota;
}}
$a = new Aluno('Carlos', 8.5);
$a->exibir();?>
```

- 6) Erro: Mesmo inválido, o valor é salvo.

```
<?php
class Pessoa {
private $idade;
public function setIdade($idade) {
if($idade < 0){
echo 'Idade inválida!';
return; }
$this->idade = $idade; }
public function getIdade(){
return $this->idade; }}
$p = new Pessoa();
$p->setIdade(-5);?>
```

- 7) Erro linha 15: \$c = Carro('Fusca'); Falta o new.

```
<?php
class Carro {
private $modelo;
public function __construct($modelo){
$this->modelo = $modelo;}
public function getModelo(){
return $this->modelo;}}
$c = new Carro('Fusca');
echo $c->getModelo();?>
```

8)

```
<?php
```

```
class Livro {

    private $titulo;
    private $autor;
    private $paginas;

    public function __construct($titulo, $autor, $paginas){
        $this->titulo = $titulo;
        $this->autor = $autor;
        $this->paginas = $paginas;
    }

    public function getTitulo(){
        return $this->titulo;
    }

    public function setTitulo($titulo){
        $this->titulo = $titulo;
    }

    public function getAutor(){
        return $this->autor;
    }

    public function setAutor($autor){
        $this->autor = $autor;
    }

    public function getPaginas(){
        return $this->paginas;
    }

    public function setPaginas($paginas){
        $this->paginas = $paginas;
    }

    public function exhibir(){
        echo $this->titulo . ' - ' . $this->autor . ' - ' . $this->paginas . '<br>';
    }
}
```

```
}
```

```
$l1 = new Livro('Dom Casmurro', 'Machado', 300);
```

```
$l2 = new Livro('1984', 'George Orwell', 250);
```

```
$l1->exibir();
```

```
$l2->exibir();
```

```
?>
```

9)

```
<?php
```

```
class ContaBancaria {
```

```
    private $titular;
```

```
    private $saldo;
```

```
    public function __construct($titular){
```

```
        $this->titular = $titular;
```

```
        $this->saldo = 0;
```

```
    }
```

```
    public function depositar($valor){
```

```
        if($valor > 0){
```

```
            $this->saldo += $valor;
```

```
        }
```

```
    }
```

```
    public function sacar($valor){
```

```
        if($valor > $this->saldo){
```

```
            echo 'Saldo insuficiente';
```

```
            return;
```

```
        }
```

```
        $this->saldo -= $valor;
```

```
    }
```

```
    public function getSaldo(){
```

```
        return $this->saldo;
```

```
}
```

```
public function getTitular(){  
    return $this->titular;  
}  
}
```

```
$c = new ContaBancaria('Ana');
```

```
$c->depositar(500);
```

```
$c->sacar(200);
```

```
echo $c->getSaldo();
```

```
?>
```

10)

```
<?php
```

```
class Aluno {
```

```
    private $nome;  
    private $nota1;  
    private $nota2;
```

```
    public function __construct($nome, $nota1, $nota2){  
        $this->nome = $nome;  
        $this->nota1 = $nota1;  
        $this->nota2 = $nota2;  
    }
```

```
    public function calcularMedia(){  
        return ($this->nota1 + $this->nota2) / 2;  
    }
```

```
    public function situacao(){  
  
        if($this->calcularMedia() >= 5){  
            echo 'Aprovado';  
        } else {  
            echo 'Reprovado';  
        }  
    }
```

```
    }  
  }  
}
```

```
$a = new Aluno('Carlos', 7, 8);
```

```
echo $a->calcularMedia() . '<br>';
```

```
$a->situacao();
```

```
?>
```

11)

```
<?php
```

```
class Retangulo {
```

```
    private $largura;
```

```
    private $altura;
```

```
    public function __construct($largura, $altura){
```

```
        if($largura > 0 && $altura > 0){
```

```
            $this->largura = $largura;
```

```
            $this->altura = $altura;
```

```
        }
```

```
    }
```

```
    public function calcularArea(){
```

```
        return $this->largura * $this->altura;
```

```
    }
```

```
    public function calcularPerimetro(){
```

```
        return 2 * ($this->largura + $this->altura);
```

```
    }
```

```
}
```

```
$r1 = new Retangulo(10, 5);
```

```
$r2 = new Retangulo(4, 4);
```

```
if($r1->calcularArea() > $r2->calcularArea()){
```

```
echo 'Retangulo 1 maior';
} else {
echo 'Retangulo 2 maior';
}
```

```
?>
```

12)

```
<?php
```

```
class Funcionario {
```

```
private $nome;
private $cargo;
private $salario;
```

```
public function __construct($nome, $cargo, $salario){
$this->nome = $nome;
$this->cargo = $cargo;
$this->salario = $salario;
}
```

```
public function aumentarSalario($percentual){
```

```
$this->salario = $this->salario * (1 + $percentual / 100);
}
```

```
public function exibir(){
```

```
echo $this->nome . '<br>';
echo $this->cargo . '<br>';
echo number_format($this->salario, 2, ',', '.') . '<br><br>';
}
}
```

```
$f = new Funcionario('Joao', 'Programador', 3000);
```

```
$f->exibir();
```

```
$f->aumentarSalario(15);
```

```
$f->exibir();
```


?>

13)

<?php

```
class Temperatura {

private $celsius;

public function __construct($celsius){
    $this->celsius = $celsius;
}

public function paraFahrenheit(){
    return ($this->celsius * 9/5) + 32;
}

public function paraKelvin(){
    return $this->celsius + 273.15;
}
}

$t1 = new Temperatura(0);
$t2 = new Temperatura(100);
$t3 = new Temperatura(37);

echo $t1->paraFahrenheit() . '<br>';
echo $t2->paraFahrenheit() . '<br>';
echo $t3->paraFahrenheit() . '<br>';

?>
```

14)

<?php

```
class Estoque {

private $produtos = [];
```

```
public function adicionarProduto($nome, $quantidade){
```

```
    $this->produtos[] = [  
        'nome' => $nome,  
        'qtd' => $quantidade  
    ];  
}
```

```
public function listarProdutos(){
```

```
    foreach($this->produtos as $p){
```

```
        echo $p['nome'] . ': ' . $p['qtd'] . '<br>';  
    }  
}
```

```
public function totalItens(){
```

```
    $total = 0;
```

```
    foreach($this->produtos as $p){
```

```
        $total += $p['qtd'];  
    }
```

```
    return $total;
```

```
    }  
}
```

```
$e = new Estoque();
```

```
$e->adicionarProduto('Mouse', 5);  
$e->adicionarProduto('Teclado', 3);  
$e->adicionarProduto('Monitor', 2);
```

```
$e->listarProdutos();
```

```
echo 'Total: ' . $e->totalItens();
```

```
?>
```

```
<?php
```

```
class Contato {
```

```
    private $nome;
```

```
    private $telefone;
```

```
    public function __construct($nome, $telefone){
```

```
        $this->nome = $nome;
```

```
        $this->telefone = $telefone;
```

```
    }
```

```
    public function getNome(){
```

```
        return $this->nome;
```

```
    }
```

```
    public function getTelefone(){
```

```
        return $this->telefone;
```

```
    }
```

```
    }
```

```
class Agenda {
```

```
    private $contatos = [];
```

```
    public function adicionarContato($nome, $telefone){
```

```
        $c = new Contato($nome, $telefone);
```

```
        $this->contatos[] = $c;
```

```
    }
```

```
    public function listar(){
```

```
        foreach($this->contatos as $c){
```

```
            echo $c->getNome() . ' - ' . $c->getTelefone() . '<br>';
```

```
        }
```

```
    }
```

```
    }
```

```
$a = new Agenda();
```

```
$a->adicionarContato('Ana', '9999-9999');  
$a->adicionarContato('Carlos', '8888-8888');  
$a->adicionarContato('Maria', '7777-7777');
```

```
$a->listar();
```

```
?>
```